



UNIVERSIDAD DE GRANADA

**Escuela Técnica Superior de Ingenierías Informática
y Telecomunicaciones(ETSIIT).
Grado en Ingeniería Informática.**

Asignatura:Periféricos y Dispositivos de Interfaz
Humana

Curso Académico 2025/26

Alumno: Carlos Alberto Rodríguez Ferreira.

Práctica 5. Experimentación con el sistema de salida de sonido.

1- Crear dos ficheros de sonido.....	2
2- Leer los sonidos y dibujar la forma de onda.....	2
3 - Información de las cabeceras de los sonidos.....	3
4 - Unir ambos sonidos.....	4
5 - Dibujar y reproducir el sonido resultante.....	4
6 - Almacenar el sonido resultante.....	5
7 - Aplicar filtro de frecuencia.....	5
8 - Aplicar eco y escucharlo del revés.....	5

1- Crear dos ficheros de sonido

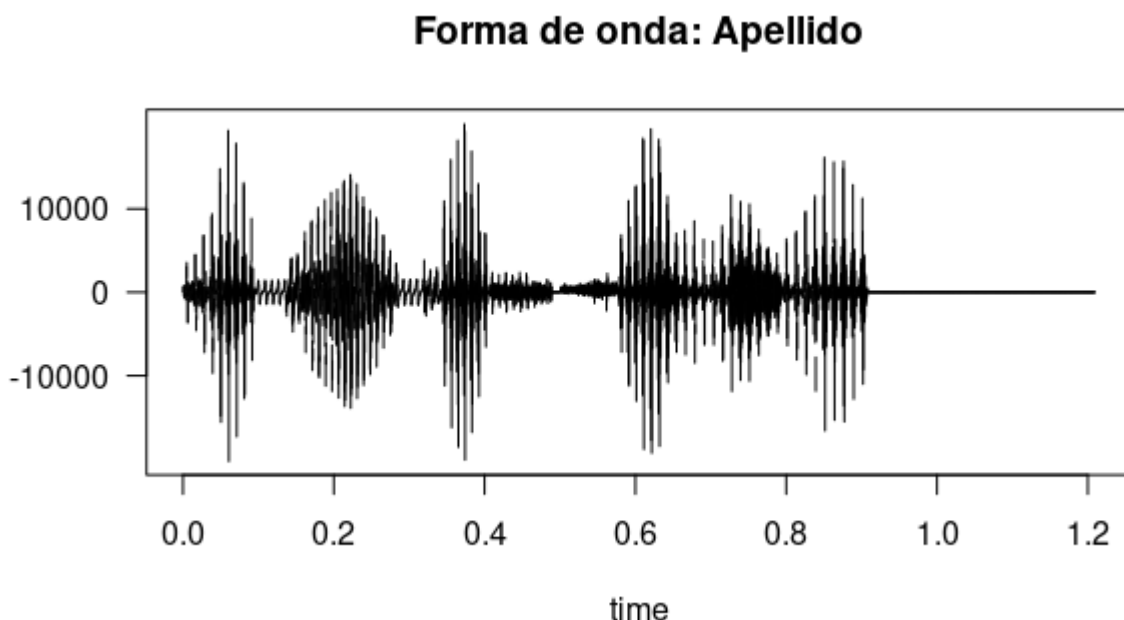
El sistema operativo que voy a utilizar durante la realización de esta practica será Linux, Ubuntu, por lo que para poder crear los dos ficheros usaré el comando espeak y el modificador -w para que sepa que no quiero que suene por los altavoces, sino que, simplemente lo guarde.

```
carlos@LordBook:~$ espeak "Carlos Alberto" -w nombre.wav  
espeak "Rodríguez Ferreira" -w apellido.wav
```

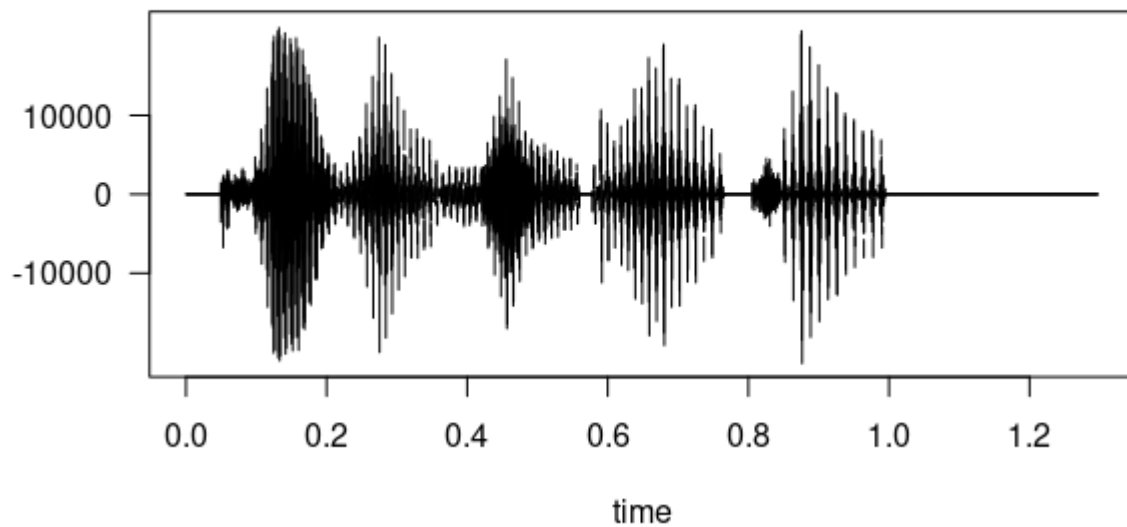
2- Leer los sonidos y dibujar la forma de onda.

Para poder leer los sonidos haré uso readWave(), esta funcion carga los archivos directamente en la memoria del programa como si fueran objetos manipulables. Para la forma de onda lo he hech con plot() pasando la variable que he usado al cargar con readwave además. he usado el parametro main=" " para que tenga un titulo a la hora de ver la onda.

```
#Ej 2  
#Carga de archivos  
nombre <- readWave("nombre.wav")  
apellido <- readWave("apellido.wav")  
  
#Forma de onda  
plot(apellido, main="Forma de onda: Apellido")  
plot(nombre, main="Forma de onda: Nombre")
```



Forma de onda: Nombre



3 - Información de las cabeceras de los sonidos

Para mostrar la información de las cabeceras he usado la función `str()` que muestra de forma compacta la información. Al ejecutarlo nos devuelve la frecuencia de muestreo en Hz, la resolución en bits y el número de canales, mono o estereo.

```
15 str(nombre)
16 str(apellido)
17
```

13:6 (Top Level) ↕

Console	Terminal ×	Background Jobs ×
R 4.1.2 · ~/		

```
> str(nombre)
Formal class 'Wave' [package "tuneR"] with 6 slots
..@ left      : int [1:28601] 0 0 0 0 0 0 0 0 0 ...
..@ right     : num(0)
..@ stereo    : logi FALSE
..@ samp.rate: int 22050
..@ bit       : int 16
..@ pcm       : logi TRUE
> str(apellido)
Formal class 'Wave' [package "tuneR"] with 6 slots
..@ left      : int [1:26665] 46 95 142 188 232 276 318 360 401 442 ...
..@ right     : num(0)
..@ stereo    : logi FALSE
..@ samp.rate: int 22050
..@ bit       : int 16
..@ pcm       : logi TRUE
```

4 - Unir ambos sonidos

Para poder unir o concatenar varios sonidos haré uso de la función `pasteq()` que sirve para concatenar. Además, tendrá el parámetro `output="Wave"` para indicar que queremos que devuelva el resultado como un archivo de sonido.

```
18 #Ej 4
19 #unir sonidos
20 nombre_completo <- pasteq(apellido, nombre, output="Wave")
21 |
```

21:1 (Top Level) ⚡

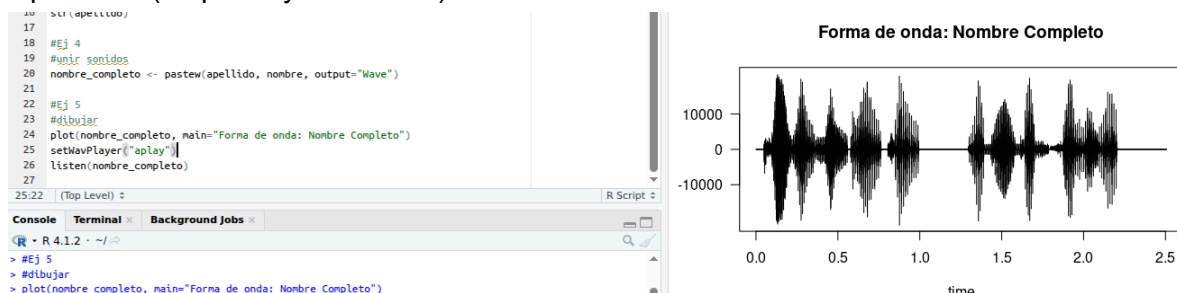
Console Terminal × Background Jobs ×

R · R 4.1.2 · ~/

```
> #Ej 4
> #unir sonidos
> nombre_completo <- pasteq(apellido, nombre, output="Wave")
```

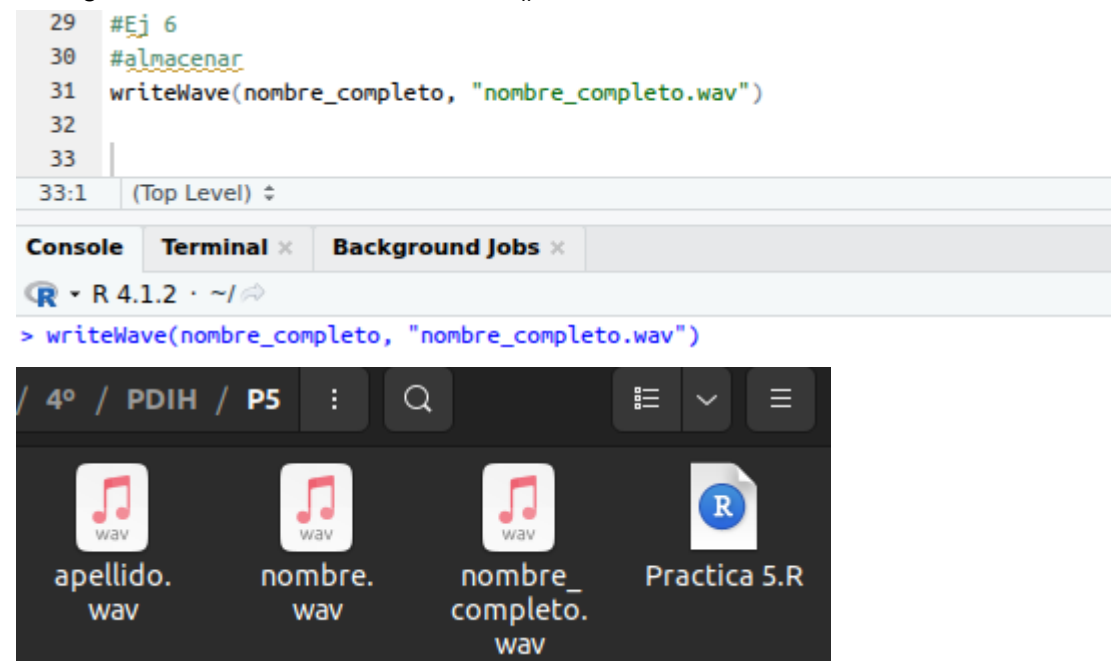
5 - Dibujar y reproducir el sonido resultante

Para dibujar el sonido resultante de la fusión de nombre y apellido, usaré `plot()` como en el ej2. Para poder escucharlo usaré `listen()`, pero, antes de usarlo usaré `setWavPlayer("aplay")` ya que me daba acceso denegado y así es la única forma de poder hacer que el sonido se reproduzca (Le pedí ayuda a la IA).



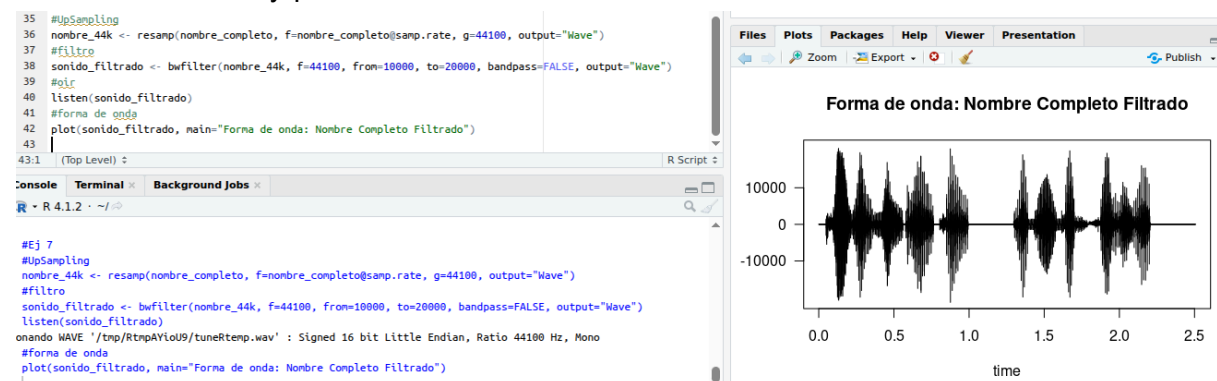
6 - Almacenar el sonido resultante

Para guardar uso la funcion writeWave()



7 - Aplicar filtro de frecuencia

Para aplicar un filtro y eliminar las frecuencias entre 10000Hz y 22000Hz voy a usar la funcion `bwfilter()` y lo configuraré con el parametro `bandpass=FALSE` para que elimine todas las muestras de audio que se encuentren en ese rango. Pero, antes de todo. Vamos a hacer un Upsampling, que es subir la frecuencia a 44100Hz para poder tener margen y evitar el error del límite de Nyquist al filtrar hasta los 20000Hz.



8 - Aplicar eco y escucharlo del revés

Para poder aplicarle eco he usado `echo()`. con el modificador `delay=c(1,2,3)` he puesto desfases de 1, 2 y 3 segundos. `amp=c(0.8,0.4,0.2)` reduce progresivamente el volumen de cada clon para crear el efecto eco. He guardado el archivo como `eco.wav`

```
#Ej 8
#Eco
sonido_eco <- echo(nombre_completo, f=nombre_completo@samp.rate, amp=c(0.8, 0.4, 0.2), delay=c(1
listen(sonido_eco)
writeWave(sonido_eco, "eco.wav")
```

Para poder hacer del reves he usado revw() para girar el audio y que el final esté al inicio y el inicio al final

```
sonido_al_reves <- revw(nombre_completo, output="Wave")
listen(sonido_al_reves)
writeWave(sonido_al_reves, "alreves.wav")
```